Material de apoyo para la clase 2: Introducción a la POO

Clases, objetos, atributos e instancias

Indicaciones: este material resume lo que se explica en la clase 2. Léelo antes de la clase y úsalo como consulta mientras haces las prácticas. Todos los ejemplos están probados en Java: escríbelos tú mismo en el IDE, sin IA, y antes de ejecutarlos anota la salida que esperas.

Hoja 1: ¿Qué es la Programación Orientada a Objetos?

La **Programación Orientada a Objetos (POO)** es un paradigma de programación que organiza el programa en **objetos**. Cada objeto junta sus datos (**atributos**) y las acciones que puede hacer (**métodos**), igual que las cosas de la vida real.

Un **paradigma** es una forma de pensar y organizar un programa.

	Programación estructurada	**Programación orientada a objetos**
Datos y acciones	Van por separado	Van juntos dentro de cada objeto
Cómo se ve el programa	Una secuencia de pasos	Objetos que colaboran entre sí
Ejemplo	Lo de la sesión 1: variables, if y for	Una clase Estudiante con sus objetos

El problema que resuelve la POO

Para guardar el nombre y el promedio de 5 estudiantes con variables sueltas se necesitan 10 variables:

```
String nombre1 = "Ana";    double promedio1 = 8.7;
String nombre2 = "Luis";   double promedio2 = 9.1;
String nombre3 = "María";  double promedio3 = 7.5;
String nombre4 = "Pedro";  double promedio4 = 6.8;
String nombre5 = "Sofía";  double promedio5 = 9.4;
```

Si además se quiere guardar la edad, ya son 15 variables. Y nada une a nombre1 con promedio1, salvo el número que les pusimos. Con la POO, cada estudiante es un solo objeto que guarda juntos su nombre, su promedio y su edad.

Lenguajes orientados a objetos: Java, C#, C++, Python y Kotlin, entre otros.

Hoja 2: Objetos

Un **objeto** es un elemento concreto que existe dentro del programa. Todo objeto tiene tres cosas:

Característica	**Qué es**	**Ejemplo con tu celular**
Estado	Los valores que tienen sus atributos en este momento	Negro, 128 GB, batería al 70 %
Comportamiento	Las acciones que puede realizar: sus	Llamar, enviar un mensaje, tomar una foto

Característica	Qué es	Ejemplo con tu celular
	métodos	
Identidad	Cada objeto es único, aunque otro tenga los mismos valores	Tu celular no es el de tu compañero, aunque sean del mismo modelo

Cómo se piensa en objetos: cualquier cosa se describe por cómo es y por lo que puede hacer.

Un auto	
Atributos (cómo es)	color: rojo; marca: Chevrolet; kilometraje: 12000 km; velocidad: 0 km/h
Métodos (qué hace)	encender(), acelerar(), frenar(), apagar()

Más objetos de la vida real: tu mochila, el auto de tu familia, la cuenta bancaria de Ana, el estudiante Luis.

Hoja 3: Clases

Una **clase** es el molde o plano que describe qué atributos y qué métodos tendrán sus objetos. **La clase no es un objeto: solo lo describe.** A partir de una clase se pueden crear muchos objetos.

Analogía	Clase	Objetos
Galletas	El molde	Cada galleta
Casas	El plano	Cada casa construida
Matrícula	La ficha en blanco	Cada ficha llena de un estudiante

Clase vs objeto

Aspecto	Clase	Objeto
Qué es	Un molde: la descripción de cómo son y qué hacen	Un elemento concreto creado a partir del molde
Cuántos hay	Una sola por cada tipo	Todos los que se necesiten
Cuándo aparece	Se escribe en el código, en un archivo .java	Se crea al ejecutar el programa, con new
Datos	Dice qué atributos existen, sin valores de un caso real	Guarda sus propios valores en cada atributo
Ejemplo	Estudiante	ana, luis, maria

Regla rápida: si puedes preguntar "¿cuál?", es un objeto. Si preguntas "¿cómo son todos?", es una clase.

Una clase en Java

```
public class Estudiante {
    // Atributos
    String nombre;
    int edad;
    double promedio;
    boolean matriculado;
}
```

Parte	Qué significa
public class	Palabras que declaran una clase pública
Estudiante	Nombre de la clase en PascalCase (cada palabra con mayúscula inicial). El archivo debe llamarse Estudiante.java
{ }	Todo lo que pertenece a la clase va entre estas llaves
String nombre;	Cada atributo: tipo, nombre en camelCase y punto y coma

Hoja 4: Atributos

Un **atributo** es una característica de un objeto. Se declara en la clase con un **tipo** y un **nombre**, y cada objeto guarda **su propio valor** en ese atributo.

Atributo	Tipo	Valor en el objeto ana	Valor en el objeto luis
nombre	String	"Ana"	"Luis"
edad	int	19	20
promedio	double	8.7	9.1
matriculado	boolean	true	false

Los dos objetos tienen los mismos atributos, pero cada uno con sus propios valores.

Valores por defecto

Si a un atributo no se le asigna un valor, Java le pone uno por defecto según su tipo:

Tipo del atributo	Valor por defecto
int	0
double	0.0
boolean	false
char	'\u0000' (carácter nulo)
String (y cualquier objeto)	null (no apunta a nada)

```
Estudiante x = new Estudiante();
System.out.println(x.nombre);       // null
System.out.println(x.edad);         // 0
System.out.println(x.promedio);     // 0.0
System.out.println(x.matriculado);  // false
```

Ojo: esto solo pasa con los atributos. Una variable local, por ejemplo una declarada dentro de main, debe tener un valor antes de usarse o el programa no compila.

Hoja 5: Instancias y el operador punto

Instanciar es crear un objeto a partir de una clase. El objeto creado se llama **instancia** de esa clase. Objeto e instancia son lo mismo: "instancia" resalta de qué clase salió.

```
Estudiante a = new Estudiante();
```

Parte	Qué significa
Estudiante	Tipo de la variable: la clase
a	Nombre de la referencia, la variable que señala al objeto
new	Crea el objeto en memoria
Estudiante()	Constructor: arma el objeto nuevo

Se lee así: "crea un nuevo Estudiante y guárdalo en a". Cada vez que se escribe new aparece un objeto distinto.

El operador punto

Con el punto se entra a un objeto para usar sus atributos y métodos: objeto.atributo u objeto.metodo().

```
Estudiante a = new Estudiante();

// Asignar valores (escribir)
a.nombre = "Ana";
a.edad = 19;
a.promedio = 8.7;

// Leer valores
System.out.println(a.nombre + " : " + a.promedio);
```

Salida:

```
Ana : 8.7
```

Resolviendo el problema de los 5 estudiantes

```
Estudiante e1 = new Estudiante();
e1.nombre = "Ana";    e1.promedio = 8.7;
Estudiante e2 = new Estudiante();
e2.nombre = "Luis";   e2.promedio = 9.1;
Estudiante e3 = new Estudiante();
e3.nombre = "María";  e3.promedio = 7.5;
Estudiante e4 = new Estudiante();
e4.nombre = "Pedro";  e4.promedio = 6.8;
Estudiante e5 = new Estudiante();
e5.nombre = "Sofía";  e5.promedio = 9.4;

System.out.println(e1.nombre + " : " + e1.promedio);
System.out.println(e2.nombre + " : " + e2.promedio);
System.out.println(e3.nombre + " : " + e3.promedio);
System.out.println(e4.nombre + " : " + e4.promedio);
System.out.println(e5.nombre + " : " + e5.promedio);
```

Salida:

```
Ana : 8.7
Luis : 9.1
María : 7.5
Pedro : 6.8
Sofía : 9.4
```

Ahora el nombre y el promedio de cada estudiante viajan juntos dentro de un mismo objeto. Si mañana se agrega la edad, basta con una línea nueva en la clase.

Hoja 6: Referencias y objetos en memoria

La variable no guarda el objeto: guarda una **referencia**, una especie de flecha que señala al objeto en memoria.

```
Estudiante a = new Estudiante();
a.promedio = 8.7;

Estudiante b = new Estudiante();
b.promedio = 8.7;

Estudiante c = a;   // no usa new
c.promedio = 9.5;

System.out.println(a.promedio);  // 9.5
System.out.println(a == c);      // true
System.out.println(a == b);      // false
```

Línea	Objeto 1	Objeto 2	Referencias
a = new Estudiante()	promedio = 0.0	(no existe)	a → Objeto 1
a.promedio = 8.7	promedio = 8.7	(no existe)	sin cambios
b = new Estudiante()	promedio = 8.7	promedio = 0.0	b → Objeto 2
b.promedio = 8.7	promedio = 8.7	promedio = 8.7	sin cambios
c = a	promedio = 8.7	promedio = 8.7	c → Objeto 1
c.promedio = 9.5	promedio = 9.5	promedio = 8.7	sin cambios

Conclusiones:

- c = a no copia el objeto: copia la flecha. a y c apuntan al mismo objeto, por eso al cambiar c.promedio también cambia a.promedio.
- a == b da false: son dos objetos distintos aunque tengan el mismo promedio (identidad).
- a == c da true: es el mismo objeto.

Hoja 7: Métodos

Un **método** es una acción que puede realizar un objeto. Se escribe dentro de la clase y puede usar y cambiar los atributos de ese objeto.

```
public class Auto {
    String marca;
    int velocidad;

    void acelerar(int cantidad) {
        velocidad = velocidad + cantidad;
    }
```

```
    void frenar() {
        velocidad = 0;
    }
}

Auto miAuto = new Auto();
miAuto.marca = "Chevrolet";
miAuto.acelerar(40);
miAuto.acelerar(20);
System.out.println(miAuto.velocidad);  // 60
miAuto.frenar();
System.out.println(miAuto.velocidad);  // 0
```

Parte del método	Qué significa
void	El método no devuelve ningún valor
acelerar	Nombre del método, en camelCase y normalmente un verbo
(int cantidad)	Parámetro: el dato que recibe el método
{ ... }	Las instrucciones que se ejecutan al llamarlo
miAuto.acelerar(40)	Llamada al método: se usa el punto y los paréntesis

Hoja 8: Los 4 pilares de la POO

1. Abstracción

Abstraer es quedarse solo con las características y acciones que importan para resolver el problema, e ignorar los detalles que no hacen falta.

- **Cómo se usa:** al diseñar una clase decides qué atributos y qué métodos poner. Lo que no sirve al problema, no entra.
- **En la vida real:** usas el control remoto con sus botones sin saber cómo están hechos sus circuitos.

	Sistema académico	Consultorio médico	Gimnasio
Clase	Estudiante	Paciente	Socio
Atributos	nombre, cédula, carrera, promedio	nombre, cédula, tipo de sangre, alergias	nombre, peso, estatura, plan
No le interesa	tipo de sangre, peso	carrera, promedio	carrera, alergias

2. Encapsulamiento

Encapsular es proteger los datos de un objeto para que solo se puedan leer o cambiar a través de sus métodos, que controlan que los cambios sean válidos.

- **Cómo se usa en Java:** los atributos se declaran private y se ofrecen métodos public para usarlos.
- **En la vida real:** en un cajero automático no tocas el dinero de la bóveda; pides retirar y el cajero revisa tu clave y tu saldo.

```java
public class CuentaBancaria {
    private double saldo;

    public void depositar(double monto) {
        if (monto > 0) {
            saldo = saldo + monto;
        }
    }

    public double getSaldo() {
        return saldo;
    }
}
```

```java
CuentaBancaria cuenta = new CuentaBancaria();
cuenta.depositar(100);
cuenta.depositar(-50);                  // se ignora
System.out.println(cuenta.getSaldo()); // 100.0
// cuenta.saldo = -500;  ERROR: saldo has private access
```

Modificador	Desde dónde se puede usar
private	Solo dentro de la misma clase
(sin modificador)	Clases del mismo paquete
protected	Mismo paquete y subclases
public	Desde cualquier clase

3. Herencia

La herencia permite crear una clase nueva a partir de otra. La clase hija (subclase) recibe los atributos y métodos de la clase padre (superclase) y agrega los suyos.

- **Cómo se usa:** cuando hay una relación "es un": un Estudiante es una Persona. En Java se escribe con extends.
- **En la vida real:** estudiantes y profesores son personas; todos tienen nombre y cédula, pero el estudiante tiene carrera y el profesor tiene título.

```java
public class Persona {
    String nombre;
    String cedula;

    void saludar() {
        System.out.println("Hola, soy " + nombre);
    }
}
```

```java
public class Estudiante extends Persona {
    String carrera;
}
```

```java
Estudiante e = new Estudiante();
e.nombre = "Ana";        // heredado de Persona
e.carrera = "Software";  // propio de Estudiante
e.saludar();             // Hola, soy Ana
```

Reglas: en Java una clase solo puede heredar de una clase padre; lo que es private en la clase padre no se usa directamente en la hija; toda clase hereda de Object aunque no se escriba.

4. Polimorfismo

Polimorfismo significa "muchas formas": el mismo mensaje, es decir, llamar al mismo método, produce un comportamiento distinto según el objeto que lo recibe.

- **Cómo se usa:** la clase padre define un método y cada hija lo sobrescribe con su propia versión usando @Override.
- **En la vida real:** presionas el mismo botón "reproducir" y suena una canción, se ve un video o empieza un podcast.

```
public class Animal {
    void hacerSonido() {
        System.out.println("...");
    }
}
public class Perro extends Animal {
    @Override
    void hacerSonido() {
        System.out.println("Guau");
    }
}
public class Gato extends Animal {
    @Override
    void hacerSonido() {
        System.out.println("Miau");
    }
}
Animal a1 = new Perro();
Animal a2 = new Gato();
a1.hacerSonido();  // Guau
a2.hacerSonido();  // Miau
```

a1 es de tipo Animal, pero el objeto es un Perro: Java ejecuta el método del objeto real.

También se habla de polimorfismo con la **sobrecarga**: varios métodos con el mismo nombre y distintos parámetros en la misma clase, por ejemplo area(double radio) y area(double base, double altura).

Resumen de los 4 pilares

Pilar	Idea clave	En Java	En la vida real
Abstracción	Quedarse solo con lo importante	Elegir los atributos y métodos de la clase	Control remoto
Encapsulamiento	Proteger los datos del objeto	private + get / set	Cajero automático
Herencia	Reutilizar una clase para crear otra	extends	Estudiante y Profesor son Persona

Pilar	Idea clave	En Java	En la vida real
Polimorfismo	Mismo mensaje, distinto resultado	@Override	Botón reproducir

Hoja 9: Diferencias y errores comunes

Concepto	Qué es	Pregunta que responde	Ejemplo
Clase	Un molde que describe atributos y métodos	¿Cómo son y qué hacen todos?	Estudiante
Objeto	Un elemento concreto con sus propios valores	¿Cuál es, exactamente?	Ana, 19 años, 8.7
Instancia	Un objeto visto como creado a partir de su clase	¿De qué clase salió?	ana es instancia de Estudiante
Herencia	Una relación entre dos clases, no un objeto	¿De qué clase más general viene?	Estudiante hereda de Persona

La clase y el objeto son "cosas"; la herencia no es una cosa, es la forma en que una clase se conecta con otra.

Errores comunes

Error	Por qué está mal
Decir "creé la clase Ana"	Ana es un objeto; la clase es Estudiante
Estudiante a = null; a.nombre = "Ana";	La referencia no apunta a ningún objeto: produce NullPointerException
Creer que c = a copia el objeto	Solo copia la referencia: los dos apuntan al mismo objeto
Guardar public class Estudiante en Main.java	El archivo debe llamarse igual que la clase pública o no compila
Pensar que la herencia crea objetos	La herencia une clases; los objetos se crean con new
Dejar los atributos public por costumbre	Cualquiera puede poner un valor inválido, como un saldo negativo

Hoja 10: Glosario

Término	Definición
Paradigma	Forma de pensar y organizar un programa
POO	Paradigma que organiza el programa en objetos que juntan datos y acciones
Objeto	Elemento concreto del programa con estado, comportamiento e identidad
Clase	Molde que describe los atributos y métodos de sus objetos
Atributo	Característica de un objeto; se declara con tipo y nombre
Método	Acción que puede realizar un objeto
Instancia	Objeto creado a partir de una clase
Instanciar	Crear un objeto con new

Término	Definición
Referencia	Variable que señala a un objeto en memoria
null	Valor de una referencia que no apunta a ningún objeto
Operador punto	Permite usar los atributos y métodos de un objeto: objeto.atributo
Constructor	Lo que se llama después de new para armar el objeto, por ejemplo Estudiante()
Abstracción	Quedarse solo con lo importante del problema
Encapsulamiento	Proteger los datos y permitir su uso solo a través de métodos
Herencia	Crear una clase a partir de otra con extends
Polimorfismo	Un mismo método con distinto comportamiento según el objeto
Sobrescritura	Volver a escribir en la clase hija un método de la padre, con @Override
Sobrecarga	Varios métodos con el mismo nombre y distintos parámetros

Referencias

[1] Oracle, "Lesson: Object-Oriented Programming Concepts," *The Java Tutorials*. Accedido: 14 oct. 2026. [En línea]. Disponible: https://docs.oracle.com/javase/tutorial/java/concepts/index.html

[2] Oracle, "Lesson: Classes and Objects," *The Java Tutorials*. Accedido: 14 oct. 2026. [En línea]. Disponible: https://docs.oracle.com/javase/tutorial/java/javaOO/index.html

[3] Oracle, "Controlling Access to Members of a Class," *The Java Tutorials*. Accedido: 14 oct. 2026. [En línea]. Disponible: https://docs.oracle.com/javase/tutorial/java/javaOO/accesscontrol.html

[4] Oracle, "Inheritance," *The Java Tutorials*. Accedido: 14 oct. 2026. [En línea]. Disponible: https://docs.oracle.com/javase/tutorial/java/IandI/subclasses.html

[5] Oracle, "Polymorphism," *The Java Tutorials*. Accedido: 14 oct. 2026. [En línea]. Disponible: https://docs.oracle.com/javase/tutorial/java/IandI/polymorphism.html

[6] P. Deitel y H. Deitel, *Cómo programar en Java*, 10.ª ed. Ciudad de México, México: Pearson Educación, 2016.